

OneCompiler

Upgrade

AI

C

Run

Console

I/O

AI Agent

166 ms

C Main.c

+

```
1 #include <stdio.h>
2 #include <stdio.h>
3
4 int contador(void) {
5     static int n = 0; // Variável estática: retém seu valor entre chamadas
6     return ++n;
7 }
8
9 int main(void) {
10     contador();
11     contador();
12     printf("%d\n", contador());
13     return 0;
14 }
```

b) Qual conceito explica?

Variáveis locais estáticas (Static Local Variables):

Uma variável local declarada com static em C possui alocação estática (vive no segmento de dados e não na pilha do registro de ativação).

Ela é inicializada apenas uma vez e preserva seu valor entre chamadas sucessivas do subprograma.

OneCompiler

Upgrade

AI

Rust

Run

Console

I/O

AI Agent

243 ms

HelloWorld.rs

```
1 fn dobra(v: &Vec<i32>) -> Vec<i32> {
2     v.iter().map(|x| x * 2).collect()
3 }
4
5 fn main() {
6     let v = vec![1, 2, 3];
7     let d = dobra(&v);
8     println!("{:?} {:?}", v, d);
9 }
```

```
[1, 2, 3] [2, 4, 6]
```

The screenshot shows the OneCompiler Python IDE interface. On the left, a file named 'main.py' is open, containing the following Python code:

```
1 total = 0
2 def adiciona(x):
3     total = total + x
4     return total
5
6 print(adiciona(5))
```

On the right, the 'Console' tab displays a traceback error:

```
Traceback (most recent call last):
  File "/box/main.py", line 6, in <module>
    print(adiciona(5))
    ^^^^^^^^^
  File "/box/main.py", line 3, in adiciona
    total = total + x
    ^^^^^
UnboundLocalError: cannot access local variable 'total' where it
is not associated with a value
```

Below the error message, there is a link that says 'Debug this error'.

b) Qual conceito explica?

Escopo de Variáveis Locais em Python e Atribuição:

Como há uma atribuição `total = ...` dentro de `adiciona()`, o Python classifica `total` como uma variável local em todo o escopo do subprograma.

Na hora de avaliar a expressão à direita (`total + x`), o Python tenta ler a variável local `total` que ainda não teve nenhum valor atribuído, gerando a exceção.

Solução: Declarar global `total` dentro da função para indicar que se refere à variável do escopo global.

b) Qual conceito explica?

Semântica de Posse (Ownership) e Movimento (Move) do Rust:

Ao passar `v` para a função `dobra(v)`, a posse do vetor é movida para o parâmetro formal da função.

O escopo chamador perde o acesso a `v`. Tentar acessar `v` em `println!` resulta num erro detectado em tempo de compilação.

Para consertar, seria necessário passar uma referência (`&Vec<i32>`) ou clonar o vetor.

The screenshot shows the OneCompiler Python IDE. The editor displays a Python script in `main.py` with the following code:

```
1 def adicionar(item, lista=[]):
2     lista.append(item)
3     return lista
4
5 print(adicionar(1))
6 print(adicionar(2))
```

The console on the right shows the output of the program:

```
[1]
[1, 2]
```

The status bar at the bottom indicates the IDE is ready and the terminal is active.

b) Qual conceito explica?

Valores padrão mutáveis em Python: Em Python, os valores padrão dos parâmetros formais são avaliados e criados apenas uma vez, no momento da definição da função (e não a cada chamada). Como `lista=[]` é um objeto mutável (list), a mesma lista é reaproveitada em chamadas subsequentes, acumulando as alterações.

The screenshot shows the OneCompiler Java IDE. The editor displays a Java class `Main` with the following code:

```
1 public class Main {
2
3     static void zera(int[] v, int n) {
4         v[0] = 0;
5         n = 0;
6     }
7
8     public static void main(String[] args) {
9         int[] v = {5, 5};
10        int n = 5;
11
12        zera(v, n);
13
14        System.out.println(v[0] + " " + n);
15    }
16 }
```

The console on the right shows the output of the program:

```
0 5
```

The status bar at the bottom indicates the IDE is ready and the terminal is active.

b) Qual conceito explica?

Passagem por valor em Java (Primitivos vs. Objetos/Arrays):

Em Java, tudo é passado por valor.

Para o primitivo `int n`, uma cópia do valor 5 é enviada para a função; alterar `n` dentro de `zera()` afeta apenas o parâmetro local.

Para o `array` `v`, o valor passado é uma cópia do ponteiro/referência para o objeto no `heap`. Alterar `v[0]` modifica o conteúdo do objeto original compartilhado na memória.

The screenshot shows the OneCompiler Python IDE interface. At the top, there's a header with the OneCompiler logo, an 'Upgrade' button, and buttons for 'AI', 'Python', 'Run', and a menu icon. On the right side of the header are icons for 'Save' and 'Login'. Below the header, the main editor area displays a Python file named 'main.py' with the following code:

```
1 fs = [lambda: i for i in range(3)]
2 print([f() for f in fs])
```

To the right of the code editor, the 'Console' tab is active, showing the output of the code: `[2, 2, 2]`. Above the console output, it says '138 ms' with a green checkmark icon. At the bottom of the IDE, there's a status bar showing 'Ready' and 'Terminal' tabs, along with icons for 'Light', 'Wiki', and 'Internet'.

b) Qual conceito explica?

Passagem por valor em Java (Primitivos vs. Objetos/Arrays):

Em Java, tudo é passado por valor.

Para o primitivo `int n`, uma cópia do valor 5 é enviada para a função; alterar `n` dentro de `zera()` afeta apenas o parâmetro local.

Para o `array` `v`, o valor passado é uma cópia do ponteiro/referência para o objeto no `heap`. Alterar `v[0]` modifica o conteúdo do objeto original compartilhado na memória.